System Design Document

OpenSpace and Climate Reanalyzer Interface

Team Violet:

Brianna Gannett, Brennon Tiensivu, Hunter Savarese, Matthew Keast, Emily Scott

Version Control

Version	Date Finalized
1.0	Nov 3, 2025
2.0	Nov 17, 2025

Table of Contents

1 Introduction..... 1
1.1 Purpose of This Document.....1
1.2 References.....1
2 System Architecture..... 2
2.1 Architectural Design..... 2
2.2 Decomposition Description..... 3
3 Persistent Data Design.....4
3.1 Database Descriptions.....4
3.2 File Descriptions..... 5
4 Requirements Matrix..... 11
Appendix A – Agreement Between Customer and Contractor..... 12
Appendix B – Team Review Sign-off.....13
Appendix C – Document Contributions..... 14

1 Introduction

This project, titled *OpenSpace and Climate Reanalyzer Interface,* is a comprehensive capstone project for University of Maine Computer Science students on V-Model Violets (formerly known as Team Violet). This project will be developed in collaboration with Dr. Sean Birkel, a Research Assistant Professor at the Climate Change Institute, and Sean Laatsch, the Director of the Versant Planetarium at the University of Maine. This project will also be collaborating with Dr. Carter Emmart and Micah Acinapura from the OpenSpace team. The goal of this project is to create an image map server that utilizes data from the Climate Reanalyzer, enabling its use by OpenSpace and in planetariums or other research settings.

1.1 Purpose of This Document

The purpose of this System Design Document (SDD) is to detail the architecture, components, interfaces, and data flow for the OpenSpace and Climate Reanalyzer Interface. This document will serve as a blueprint for our team, the clients, and the teaching staff for the Capstone class, outlining the functionality of the system, possible constraints, and expectations regarding performance, security, and design. The SDD is intended for:

- Capstone Team Members: To guide the group through the implementation process, as well as ensure the client's needs are met.
- Clients: To show how we plan to construct the system in a transparent and informative way
- University of Maine Faculty: To assess the progress of the project

1.2 References

The table below lists the sources being used for this project currently:

Title	Author(s)	Publisher / Organization	Date	Citation
Climate Reanalyzer	Dr. Sean Birkel	Climate Change Institute, University of Maine	Ongoing	Birkel, S. (n.d.). Weather forecasts. Climate Reanalyzer. https://climatereanalyzer.org/
OpenSpace	OpenSpace Team	American Museum of Natural History	Ongoing	OpenSpace. (n.d.). OpenSpace. https://www.openspaceproject.com
Apache	Apache Software	Open Source	1997-Curre	Apache Software

HTTPD Server Documentation	Foundation		nt	Foundation. (n.d.). Apache HTTP Server Project. https://httpd.apache.org/
Software Engineering (10th Edition)	Ian Sommerville	Pearson Education	2015	Sommerville, I. (2015). Software engineering (10th ed.). Pearson Education.

2 System Architecture

This section introduces the general architecture of the OpenSpace and Climate Reanalyzer Interface. Section 2.1 features the general architectural design (including a diagram and a brief write-up). Section 2.2 features the more detailed decomposition design. It has a diagram that is a more broken-down and specific version of the one found in 2.1, as well as a brief write-up.

2.1 Architectural Design

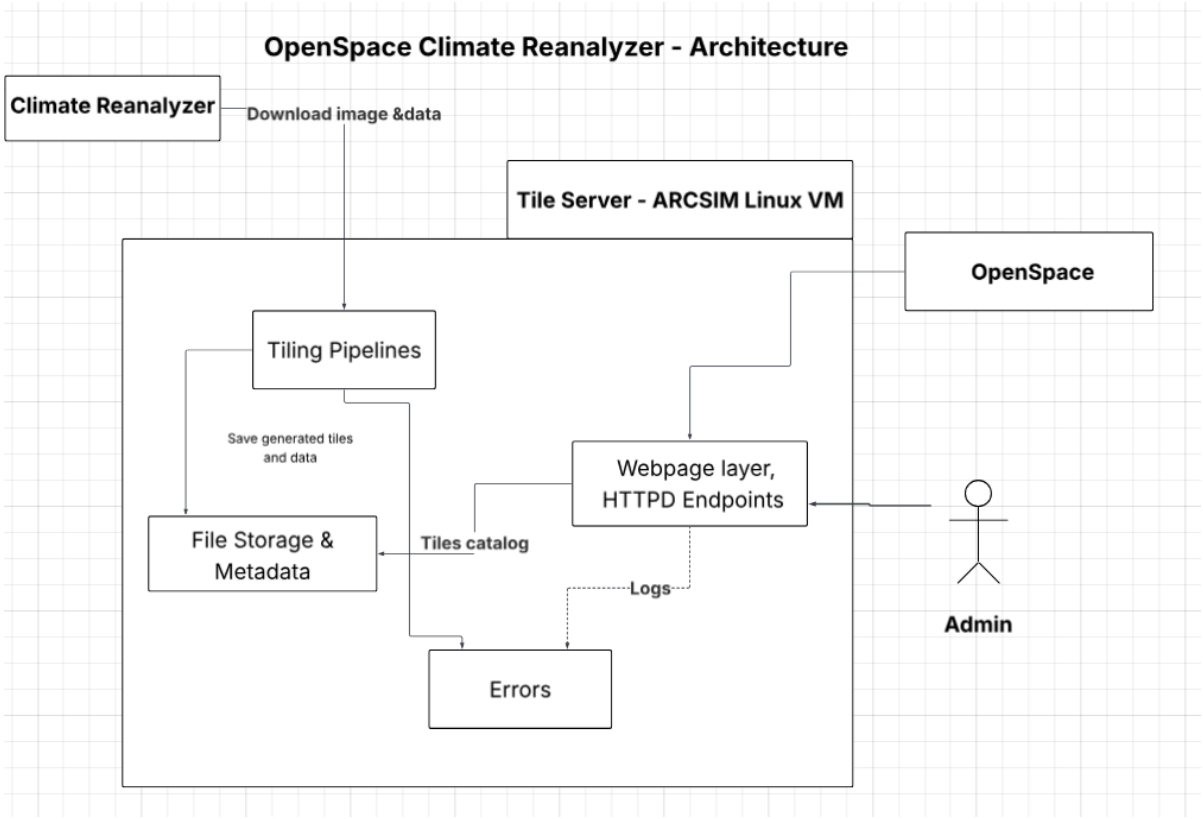

Figure 2.1 OpenSpace - Climate Reanalyzer Architecture

The OpenSpace and Climate Reanalyzer system is a client-server architecture that connects available data from the Climate Reanalyzer to the OpenSpace platform. Figure 2.1 shows the architecture and how the components interact. The goal is to show data flow between the tile server and OpenSpace's platform. This design will allow OpenSpace to request the map tiles it needs.

At the top, the Climate Reanalyzer (CR) is an external data provider. These images will be fetched by the tile server, which will convert the map images to smaller tiles using GDAL utilities. The Tile Server will expose the API via Apache HTTPD that lets OpenSpace request individual tiles or map catalogs. Administrators can also upload new images through the same API. When OpenSpace requests a new map, the server will check whether it already exists. If not, it will download the source image and generate the required tiles, then serve them to the client.

The system runs on a Linux virtual machine (ARCSIM VM) provided by the University of Maine. It will use Apache HTTPD as the web service and GDAL for the image tiling and JSON metadata handling. OpenSpace will communicate with the server via HTTPS requests to retrieve tiles and catalogs for visualization in the planetarium environment. This setup will allow efficient performance and cross-platform compatibility.

2.2 Decomposition Description

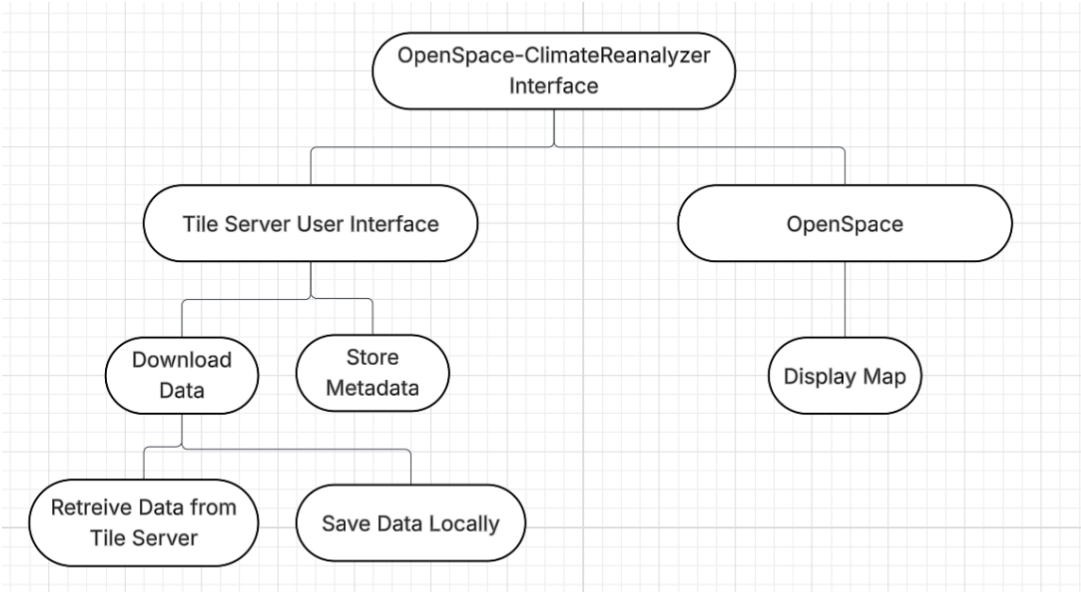

Figure 2.2 OpenSpace - Climate Reanalyzer Decomposition Diagram

This section describes the decomposition of the OpenSpace and Climate Reanalyzer Interface system we are developing. It highlights and describes a more in-depth description of what was described in section 2.1. Our system is mostly procedural, hence the structural decomposition diagram seen above (Figure 2.2).

Our system is a client-server model, which means that clients request services and servers provide them. For the OpenSpace and Climate Reanalyzer Interface, the client would be the user interface for the Tile Server that is brought up in the OpenSpace application itself. The server is

the actual tile server, which holds the maps and metadata that have been previously uploaded.

Our system's functions are broken down into two main sections: actions handled through the tile server and actions handled through OpenSpace itself. The main functions of the Tile Server are to download data to the user's local system and to store the associated metadata. The function that OpenSpace serves is to display the maps that have been saved or cached from the tile server.

3 Persistent Data Design

In this section, we will cover Database design and File Structure. The design of the database will show how we will store the data in our server and how it interacts with the interface and Openspace. Our file structure shows the direct format of the server along with datatypes. This information will be used to build our product, and can work as a guide for developers.

3.1 Database Descriptions

For our project, we need to create a Tile server that will serve as a database of maps and tiled map data. Our product will be interfacing this data with the Openspace Project to allow Openspace to load and display these maps when requested. The database should not be complex and should not require any form of login authentication, as all of the maps are publicly sourced and free to access by anyone. Access to the database will be exclusively through a browser within Openspace, and will only allow for downloading of files and requesting new maps to be added to the database from ClimateReanalyzer. Manually adding and deleting files will only be allowed through direct access to the Linux machine provided to us by Dr. Sean Birkel.

Data types in the database itself include:

- Map_id: uuid, <1Kb, The specific ID for each map.
- Rectangular_map: PNG image, 10-100Mb, The unaltered base image of the map
- Map_tile: png image, 1-10Mb, A single Tile of the map after modification
- tiled_map_file, tif compressed image, 1-10Gb, The tiled tif format compressed image
- Tiled_map_data: WMS tile image index file, 10Mb, Index file for getting each tile.
- Map_metadata: JSON, <100Kb, JSON data used to communicate with the server.

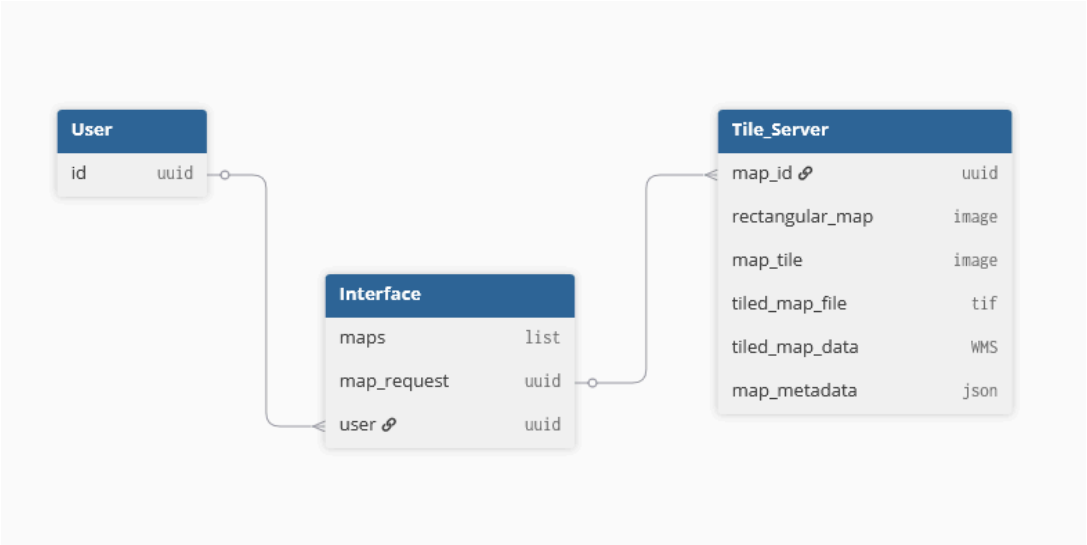

Figure 3.1 OpenSpace - Climate Reanalyzer Database Schema

3.2 File Description

File Structure:

```
/OpenSpace-x-ClimateReanalyzer/
├─ .devcontainer/
│  └─ devcontainer.json
├─ .github/
│    └─ CODEOWNERS
│    └─ workflows/
│       └─ validate-commit-message.yml
├─ webconf/
│  └─ {map_id}.webconf
├─ data/
│  ├─ Earth
│  │  ├─ Bmng
│  │  │    └─ {map_id}/{z}/{x}/{y}.idx
│  │  │    └─ {map_id}/{z}/{x}/{y}.jpg
│  │  │    └─ {map_id}/{z}/{x}/{y}.mrf
│  │  ├─ Gebco
│  │  │    └─ {map_id}/{z}/{x}/{y}.ppg
│  │  │    └─ {map_id}/{z}/{x}/{y}.png
│  │  │    └─ {map_id}/{z}/{x}/{y}.mrf
│  ├─ source_images/
│  │  └─ {map_id}.tif
├─ documents/
│  └─ {document_id}
├─ genconf.sh
├─ setup-ahtse.sh
├─ README.md
└─ .gitignore
```

Directory Summary

1. **.devcontainer/**
   Contains the Visual Studio Code development container configuration. Defines the environment, tools, and extensions required for development consistency.
2. **.github/**
   Contains repository-level configuration files used by GitHub, including:

   - **CODEOWNERS** – Defines repository ownership and review responsibilities.
   - **workflows/** – Contains CI/CD pipeline definitions (for example, commit message

validation)

3. **webconf/**
   Contains configuration files (.webconf) that define how specific datasets (maps or layers) are rendered within the visualization engine.
4. **data/**
   Stores all imagery and elevation datasets. Data is organized in a hierarchical, tile-based structure that supports multi-resolution map rendering.
5. **documents/**
   Contains project documentation, dataset metadata, or generated reports.
6. **Shell Scripts:**

   - **genconf.sh** – Generates configuration files for datasets.
   - **setup-ahtse.sh** – Initializes the Advanced Hierarchical Tile Service Engine (AHTSE) used to serve tiled raster data.

7. **README.md**
   Provides a project summary, setup instructions, and contributor information.
8. **.gitignore**
   Defines which files and directories are excluded from version control.

Data Directory Structure:

```
├─ data/
│  ├─ Earth
│  │  ├─ Bmng
│  │  │   └─ {map_id}/{z}/{x}/{y}.idx
│  │  │   └─ {map_id}/{z}/{x}/{y}.jpg
│  │  │   └─ {map_id}/{z}/{x}/{y}.mrf
│  │  ├─ Gebco
│  │  │   └─ {map_id}/{z}/{x}/{y}.ppg
│  │  │   └─ {map_id}/{z}/{x}/{y}.png
│  │  │   └─ {map_id}/{z}/{x}/{y}.mrf
│  ├─ source_images/
│  │  └─ {map_id}.tif
```

- The **data/** directory contains the geospatial raster data used by the visualization system. Files are stored by dataset and tile coordinate, allowing for efficient retrieval at multiple zoom levels.
- *A typical file path is formatted as follows*:

```
data/Earth/<DatasetName>/<MapID>/<Z>/<X>/<Y>.<extension>
```

- Where:

- **MapID** – A string identifying the dataset (for example, “BlueMarble”).
- **Z** – Integer representing the zoom level (0 = lowest resolution).
  X – Integer representing the horizontal tile coordinate.
- **Y** – Integer representing the vertical tile coordinate.

File Descriptions

1. **Index File (.idx)**
   a. **Purpose**: Holds metadata about tile positions and sizes within the associated Meta
2. **Raster Format (MRF) data file**
   a. **Purpose**: Provides structural and metadata information describing a multi-resolution raster dataset.
3. **Image Tiles (.jpg, .png, .ppg)**
   a. **Purpose**: Contain the actual raster image data used for display at specific coordinates and zoom levels.
4. **Source Imagery (.tif)**
   a. **Purpose:** High-resolution GeoTIFF source images are used to generate the tiled datasets.
5. **Web Configuration File (.webconf)**
   a. **Purpose:** Defines how datasets are displayed in the web visualization interface, including paths, zoom parameters, and projection details.
   b. **Key Fields**
      i. RegExp (string) = "*./tile/.*" - URL/path pattern that this configuration applies to.
      ii. Size (int int int int) = "4978 6322 1 3" - Raster width, height, bands, and data-type code.
      iii. PageSize (int int int int) = "512 512 1 3" - Tile width, tile height, bands, and data-type code.
      iv. DataFile (string, path) = "/<project-path>/data/Earth/Bmng/test.pjg" - Backing raster data file.
      v. IndexFile (string, path) = "/<project-path>/data/Earth/Bmng/test.idx" - Index file mapping tiles within DataFile.
      vi. SkippedLevels (int) = 0 - Number of pyramid levels omitted from serving.
      vii. Example webconf:

```
RegExp .*/tile/.*
Size 4978 6322 1 3
PageSize 512 512 1 3
DataFile /<project-path>/data/Earth/Bmng/test.pjg
IndexFile /<project-path>/data/Earth/Bmng/test.idx
SkippedLevels 0
```

6. **Development Container Configuration (devcontainer.json)**
   a. **Purpose:** Defines the local development environment used by the team.
   b. **Key Fields:**

i. name (string) = "Ubuntu" - Human-readable environment name.
ii. image (string) = "mcr.microsoft.com/devcontainers/base:noble" - Base container image.
iii. features (object)
iv. ghcr.io/devcontainers/features/git-lfs:1 (object) - Enables Git LFS.
v. ghcr.io/devcontainers-extra/features/apt-packages:1 (object)
vi. packages (string) = "gdal-bin,libgdal-dev,build-essential,cmake,apache2,apache2-dev,libapr1-dev,libaprutil1-dev,libjpeg-dev,libpng-dev,zlib1g-dev,libcurl4-openssl-dev,sqlite3,libsqlite3-dev,git" - Apt packages to install.
vii. postCreateCommand (string) = "bash setup-ahtse.sh && bash genconf.sh" - Command run after container is created.

7. **GitHub Workflow Configuration (validate-commit-message.yml)**
   a. **Purpose:** Defines automated actions used by GitHub to validate commit messages or run tests.
   b. **Key Fields:**
      i. Name: "Validate Commit Message" — Workflow name.
      ii. On: push — Event trigger.
      iii. Runs-on: "ubuntu-latest" — Operating system for the runner.
      iv. Uses: "actions/checkout@v4" — Checks out the repository.
      v. Fetch-depth: 0 — Ensures full commit history is available.
      vi. Id: "range" — Identifier for the step calculating the commit range.
      vii. Shell: bash — Execution shell for steps.
      viii. Run (range): Script that determines the “from” and “to” commit SHAs.
      ix. Name (validation): "Validate commit types" — Step that checks commit message format.
      x. VALID_TYPES: "feat|fix|chore|refactor|test|docs|ci" — Allowed commit prefixes.
      xi. Run (validation): Bash loop that validates each commit message against the allowed format.
8. **genconf.sh**
   a. **Purpose:** Scans ./webconf for .webconf files, validates that corresponding data/index files exist under the data root, fixes DataFile/IndexFile paths in-place, and generates ahtse.conf with Apache/AHTSE directory blocks for serving MRF tiles.
   b. **Key Fields:**
      i. DATA_FOLDER: /workspaces/OpenSpace-x-ClimateReanalyzer/data — Root directory for MRF data and tiles (can be overridden by environment variable).
      ii. WEBCONF_ROOT: <repo-root>/webconf — Directory containing .webconf configuration files.
      iii. OUT_CONF: <repo-root>/ahtse.conf — Output file generated for

Apache/AHTSE configuration.

iv. Required Lines: "DataFile <path>" and "IndexFile <path>" — Must be present in each .webconf file.

v. Supported Extensions: DataFile uses .ppg or .pjg; IndexFile uses .idx.

vi. Swap Detection: Automatically corrects swapped file references (e.g., .idx listed as DataFile).

vii. Fallback Search: Checks alternate directories if expected file paths are missing.

viii. Path Normalization: Rewrites .webconf entries with verified absolute file paths.

ix. Apache Block Output: Adds configuration for each dataset to ahtse.conf with:

1. Options: -Indexes -FollowSymLinks -ExecCGI
2. MRF_RegExp: */tile/.*
3. MRF_ConfigurationFile: Absolute path to the corresponding .webconf.

x. Logging and Safety: Uses strict error handling (set -euo pipefail) and detailed logging.

xi. Dependencies: Requires bash, sed, awk, grep, find, and realpath/readlink.

9. **setup-ahtse.sh**

a. **Purpose:** Automates installation of system dependencies; builds AHTSE-related libraries and Apache modules from source; registers/load-files for Apache; creates a site config exposing the project's data and including the generated ahtse.conf.

b. **Key Fields:**

i. Package Installation: Installs core dependencies, including Apache, GDAL, build tools, and development libraries.

ii. Apache Paths:

1. APXS_BIN: apxs — Apache extension tool.
2. APR_INC: APR includes the directory path.
3. AP_INC: Apache include directory path.
4. AP_LIBEXECDIR: Apache module directory path.

iii. Include Paths: Adds Apache and APR include directories to environment variables (CPATH and CPLUS_INCLUDE_PATH).

iv. Build Directories: Creates required folders under the user home directory — wms_modules, modules, lib, and include.

v. Dynamic Linker Config: Registers $HOME/lib in /etc/ld.so.conf.d/ahtse.conf and runs ldconfig.

vi. Repositories Cloned:

vii. libahtse, AHTSE, libicd, mod_mrf, mod_receive, mod_sfim, mod_reproject, mod_convert (from GitHub).

viii. Build Order: Builds and installs modules in dependency order:

ix. mod_receive → libicd → libahtse → mod_mrf → mod_convert →

mod_reproject → mod_sfim.

x. Build Configuration: Generates local Makefile.lcl files with proper include paths; uses CMake as a fallback for libicd.

xi. Installed Files:
    1. Compiled modules stored in $HOME/modules.
    2. Headers installed to $HOME/include.
    3. Some modules are also installed in Apache's module directory.

xii. Apache Module Load Files: Creates and enables .load files in /etc/apache2/mods-available for available modules (mrf, convert, receive, retile, sfim).

xiii. Site Configuration:
    1. Creates /etc/apache2/sites-available/100-ahtse.conf with:
    2. Alias /tiles/ mapped to /workspaces/OpenSpace-x-ClimateReanalyzer/data/

xiv. Includes the generated ahtse.conf file.

xv. Validation: Runs apachectl configtest and restarts Apache. Verifies loaded modules using apachectl -M.

Summary:

The OpenSpace-x-ClimateReanalyzer system relies on structured and standardized file formats to ensure interoperability across datasets, visualization components, and processing tools.
Each file type described here contains sufficient metadata and data fields for a developer to implement loading, parsing, and serving mechanisms within the system.

4 Requirements Matrix

The following table identifies which system components satisfy each of the functional requirements for the OpenSpace and Climate Reanalyzer Interface. The functional requirements are defined in more detail in the System Requirements Specification document.

Requirement Use Case # / Name	Use Case Description	System Component(s)
1: Map - Tile Conversion	Convert 2:1 map image into tiled map format and display in OpenSpace.	check_availability(map_id) download_image(CR_image) generate_tiles(image) send_tiles(tiles)

2: Download and Save Map Files	Download and locally store selected tile maps for offline use.	select_image(image) generate_tiles(image) download_tiles(tiles, path)
3: Admins Uploading Large Data Files	Upload source maps to the tile server	upload_map(map_id, path)
4: List Available CR Maps	List available Climate Reanalyzer maps and metadata	check_maps() get_matadata(maps) send_metadata(metadata)
5: Multiple tile levels	Provide inner layers of tiles and display in OpenSpace	generate_tiles(image, layer) send_tiles(tiles)
6: Animating Maps and Data	Display animation of data over a chosen time range for selected region	get_maps(map_class, time_range) generate_tiles(image) send_tiles(tiles)
7: Handling Errors and Status Feedback	Alert users upon system errors	catch(error) log(error) display(error)

Appendix A – Agreement Between Customer and Contractor

The team at V-Model Violets and our clients, Dr. Sean Birkel, Shawn Laatsch, and the team at The OpenSpace Project, all agree that the documentation provided here adequately represents how the product will be structured and its design. This document shows our plans on how things will be structured as we develop, what we believe our database and file structure will look like, and how we plan to meet the requirements we have set out.

As the product is developed, changes may be made to this document to cover additional challenges that need designing, as well as if anything is changed once more work is completed. If changes are made, the clients will be notified of the changes so they remain up to date on the design of the product.

Signed
V-Model Violets Team:

Emily Scott: Emily Scott **Date: 11/17/2025**

Brianna Gannett: Brianna Gannett **Date: 11/17/2025**

Brennon Tiensivu: Brennon Tiensivu **Date: 11/17/2025**

Hunter Savarese: Hunter Savarese **Date: 11/17/2025**

Matthew Keast: Matthew Keast **Date: 11/17/2025**

Clients:

Dr. Sean Birkel:_____ **Date:**_____/_____/_____

Shawn Laatsch:_____ **Date:**_____/_____/_____

Micah Acinapura:_____ **Date:**_____/_____/_____

Appendix B – Team Review Sign-off

We, the team at V-Model Violets, all agree and sign off on all the information provided in this document. The document has been reviewed by all members, and we agree on its content, and it is formatted in a way we all agree on. No one in the team has any major issues with the document or its layout, but they will have the chance to list some minor things they disagree with.

Signed
V-Model Violets Team:

Emily Scott: Emily Scott **Date: 11/17/2025**

Comments:_____

Brianna Gannet: Brianna Gannet **Date: 11/17/2025**

Comments:_____

Brennon Tiensivu: Brennon Tiensivu **Date: 11/17/2025**

Comments:_____

Hunter Savarese: Hunter Savarese **Date: 11/17/2025**

Comments:_____

Matthew Keast: Matthew Keast **Date: 11/17/2025**

Comments:_____

Appendix C – Document Contributions

Emily Scott's contributions: Completed section 3.1 and associated diagram. Added Appendices A - C. Assigned jobs for who would do which sections of the document. 20%

Brianna Gannet's Contributions: Completed section 2.2. 20%

Brennon Tiensivu's Contributions: Completed the Introduction and sections 1.1-1.2. Completed section 3.2. 20%

Hunter Savarese's Contributions: Completed section 2.1. 20%

Matthew Keast's Contributions: Completed section 4. 20%